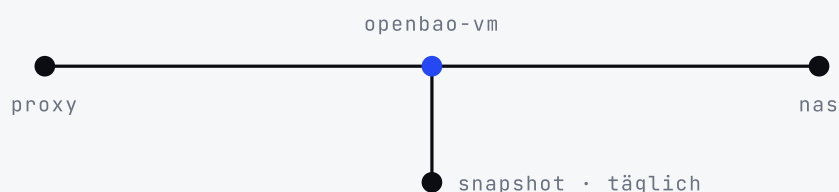


OPENBAO AUF EINER VM

Ein Knoten mit Ansible und OpenTofu: Installation, Bootstrap, Snapshots – und ein Disaster Recovery, das man geprobt hat.



OpenBao auf einer VM

Ein einzelner Knoten mit Ansible und OpenTofu: Installation, Bootstrap, Snapshots – und ein Disaster Recovery, das man geprobt hat

Thomas Zachmann

17. September 2026

Inhalt

Worum es geht

- Warum von Hand, und warum ohne KI
- Ein Wort zu den Werten in diesem Leitfaden
- Lizenz und Haftung
- Für wen
- Die Bausteine
- Zwei Schichten, sauber getrennt
- Die Architektur in einem Bild
- Warum ein Knoten und nicht drei

Teil I – Was OpenBao anders macht als Vault

Teil II – Schicht 1: Ansible

- Was die Rolle tut
- Die Konfiguration
- Die systemd-Härtung
- Deployment
- Der NFS-Export

Teil III – Schicht 2: OpenTofu

- Warum der Provider vault heißt
- Was die Konfiguration anlegt – und warum
- Die Dateien
- Welcher Wert wohin
- Der erste Lauf – mit dem Root-Token
- Jeder weitere Lauf
- Passwort rotieren

Teil IV – Das Bootstrap-Runbook

- Voraussetzungen
- 1. Mit dem Root-Token anmelden
- 2. Konfiguration mit OpenTofu anwenden
- 3. Den Ersatzzugang beweisen – nicht überspringen
- 4. Snapshot-Token – der eine Schritt, der manuell bleiben muss
- 5. Root-Token widerrufen
- 6. Abschlussprüfung

Teil V – Betrieb

- Unseal nach jedem Neustart
- Snapshots prüfen
- Einen Snapshot von Hand verifizieren
- Restore in dieselbe laufende Instanz
- Upgrade

Teil VI – Disaster Recovery: eine neue VM

- Was man haben muss

- Das, was alle falsch machen

- Ablauf

- Restore-Probe

- Kann ich wirklich wiederherstellen? – Checkliste

Teil VII – Bewusste Entscheidungen und bekannte Risiken

- Entscheidungen

- Risiken

- Troubleshooting

Anhang A – Alle Kommandos

Anhang B – Glossar

Worum es geht

Ein Secrets-Store muss zwei Dinge gleichzeitig: sicher sein und wiederkommen. Sicher, damit niemand ohne Berechtigung an Geheimnisse kommt. Wiederkommen, damit ein Plattenausfall, ein kaputtes Upgrade oder ein versehentliches `rm -rf` nicht das Ende aller Zugangsdaten ist, die im Unternehmen – oder im Homelab – je gespeichert wurden.

Dieser Leitfaden baut OpenBao als **einzelnen Knoten auf einer Ubuntu-VM**: Raft-Storage, Shamir-Unseal, TLS, systemd-Härtung, tägliche Snapshots auf ein NAS. Die Installation macht Ansible, die Konfiguration macht OpenTofu, und die wenigen Schritte, die manuell bleiben müssen, sind als Runbook beschrieben – in einer Reihenfolge, die einen Lockout ausschließt. Am Ende steht ein Disaster-Recovery-Verfahren auf eine leere VM, inklusive der Frage, wie man es probt, ohne die laufende Instanz zu gefährden.

Es ist der dritte Teil einer Reihe. N° 1 baut denselben OpenBao im Kubernetes-Cluster per Helm; N° 2 lässt eine Anwendung dynamische Datenbank-Credentials daraus beziehen. Der VM-Aufbau ist der ausgereifteste der drei – viele Entscheidungen in N° 1 sind von hier übernommen.

Warum von Hand, und warum ohne KI

Die Automatisierung in diesem Leitfaden – die Ansible-Rolle, das OpenTofu-Verzeichnis – ist das Ergebnis, nicht der Weg. Jede Zeile darin beantwortet eine Frage, die vorher von Hand gestellt wurde: Warum kein `disable_mlock`? Warum `MemorySwapMax=0`? Warum ist der Root-Token nach dem Bootstrap weg, und was passiert, wenn man ihn zu früh widerruft?

Wer eine KI bittet, „OpenBao mit Ansible zu installieren“, bekommt eine Rolle, die läuft. Er bekommt nicht die Erkenntnis, dass ein `notify: restart openbao` den Secrets-Store beim zweiten Playbook-Lauf versiegelt vom Netz nimmt. Diese Erkenntnisse sind der Inhalt dieses Leitfadens; die Rolle ist nur ihr Beleg.

Der Maßstab: Wer diesen Leitfaden durchgearbeitet hat, kann ohne Hilfe aufzeichnen, welche Schritte nach `init` in welcher Reihenfolge kommen und welcher davon nicht mehr rückgängig zu machen ist. Und er kann erklären, warum bei einem Restore auf eine neue VM zwei verschiedene Sätze Unseal-Keys nacheinander gebraucht werden.

Werkzeuge wie [nyrvex](#), das die Konfiguration von Secret Store und Identity Provider einer AI-Plattform generiert, nehmen einem diese Schritte später ab. Man sollte sie einmal selbst gegangen sein, um beurteilen zu können, was generiert wurde.

Ein Wort zu den Werten in diesem Leitfaden

Der Aufbau lief in einem Homelab. IP-Adressen, Hostnamen, der NAS-Typ und der Benutzername des Administrators sind durch Platzhalter ersetzt:

Platzhalter	Bedeutung
192.0.2.40	die OpenBao-VM
192.0.2.41	der Reverse-Proxy davor
192.0.2.160	das NAS mit dem NFS-Export /volume1/openbao-backup
bao.example.internal	der Hostname, unter dem OpenBao erreichbar ist
admin	der Benutzer, der den Root-Token ersetzt

Unseal-Keys, Root-Token, Passwörter und Passphrasen erscheinen nirgends – sie stehen auch im Original in keinem Log, weil die Schritte, die sie erzeugen, bewusst nie automatisiert werden.

Lizenz und Haftung

Dieser Leitfaden steht unter CC BY 4.0: Er darf kopiert, weitergegeben und bearbeitet werden, auch kommerziell, solange der Autor genannt wird. Er wird ohne Gewähr bereitgestellt. Alles darin wurde in einer Entwicklungsumgebung durchgeführt; wer es in einer anderen Umgebung nachvollzieht, tut das auf eigene Verantwortung.

Für wen

Für Leserinnen und Leser, die Linux-Administration, systemd und die Grundzüge von Ansible und Terraform/OpenTofu kennen und OpenBao oder HashiCorp Vault schon einmal gesehen haben. Die Konzepte – Seal/Unseal, Policies, Auth-Methoden, Audit – setzt dieser Leitfaden voraus. Wer sie von Grund auf lernen will, mit Labs, die auf dem Laptop laufen, findet das in meinem Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice).

Die Bausteine

Baustein	Version	Rolle
Ubuntu Server	24.04 LTS	die VM, 1 vCPU / 2 GB / 20 GB
OpenBao	2.6.2, .deb von GitHub Releases	der Secrets-Store
Ansible	2.17	Schicht 1: Host, Paket, TLS, systemd, Firewall, Timer
OpenTofu	≥ 1.11	Schicht 2: Audit, Policies, Auth-Methoden
Reverse-Proxy	–	terminiert TLS mit einem öffentlich gültigen Zertifikat
NAS	–	NFS-Export als zweiter Ablageort für Snapshots

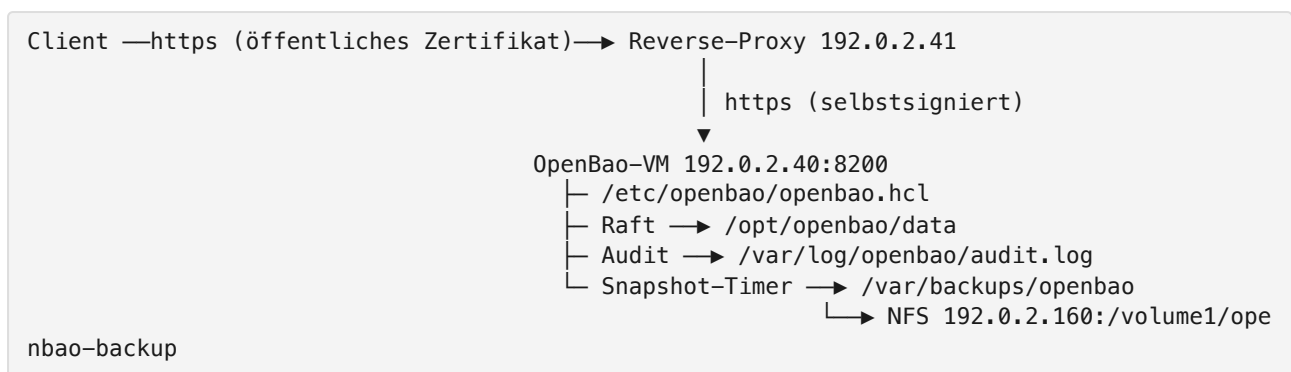
Zwei Schichten, sauber getrennt

Schicht	Was	Womit	Braucht ein OpenBao-Token?
Host	Paket, TLS, systemd, Firewall, Snapshot-Timer	Ansible	nein
Konfiguration	Audit-Device, Policies, Auth-Methoden	OpenTofu	ja

Die Trennlinie ist nicht „IaC gegen CLI“, sondern **Konfiguration gegen geheime Werte**. Die Ansible-Rolle bleibt absichtlich token-frei; alles, was ein Token braucht, lebt in OpenTofu. Geheime *Werte* liegen in keinem von beiden.

Manuell bleibt nur, was manuell bleiben muss: `init`, `unseal`, der Root-Login für das allererste `tofu apply`, und der Snapshot-Token – ein geheimer Wert, der als Datei auf der VM landen muss.

Die Architektur in einem Bild



Storage	integriertes Storage (Raft), ein Knoten
Unseal	Shamir 3 von 5, manuell
TLS außen	öffentlich gültiges Zertifikat am Reverse-Proxy
TLS innen	selbstsigniert, 10 Jahre, von der Rolle erzeugt
Audit	File-Device, <code>logrotate</code> mit <code>SIGHUP-Reload</code>
Backup	Raft-Snapshots per <code>systemd-Timer</code> , lokal + NFS

Warum ein Knoten und nicht drei

Es gibt genau **einen** physischen Host. Drei VMs darauf schützen nicht gegen den dominanten Ausfall – Host weg – und verdreifachen mit Shamir den manuellen Aufwand: 3 Knoten × 3 Keys = **9 Eingaben pro Neustart**. Schutz kommt hier aus Backup und Restore, nicht aus Quorum.

Wachstum bleibt möglich: Die Zertifikat-SANs enthalten bereits `bao-02` und `bao-03`, ein späteres `bao operator raft join` braucht kein neues Zertifikat.

Teil I – Was OpenBao anders macht als Vault

Alles Folgende wurde gegen das tatsächliche `openbao_2.6.2_linux_amd64.deb` und die OpenBao-Dokumentation geprüft, nicht per Suchen-und-Ersetzen aus einem Vault-Setup übernommen.

	Vault	OpenBao 2.6.2
Installation	APT-Repo <code>apt.releases.hashicorp.com</code>	kein APT-Repo – <code>.deb</code> von GitHub Releases, hier gegen eine signierte <code>checksums.txt</code> verifiziert
Paket / Binary	<code>vault / vault</code>	<code>openbao / bao</code>
Config	<code>/etc/vault.d/vault.hcl</code>	<code>/etc/openbao/openbao.hcl</code> (eine Paket-Conffile)
Daten / TLS	<code>/opt/vault/{data,tls}</code>	<code>/opt/openbao/{data,tls}</code>
Service / User	<code>vault.service / vault</code>	<code>openbao.service / openbao</code>
CLI-Variablen	<code>VAULT_ADDR</code> , <code>VAULT_CACERT</code>	<code>BAO_ADDR</code> , <code>BAO_CACERT</code> (Token-Helper schreibt weiter <code>~/.vault-token</code>)
<code>disable_mlock</code>	mit Raft empfohlen <code>true</code>	weg – <code>mlock</code> entfernt, ersetzt durch <code>MemorySwapMax=0</code>
Unauthentifizierte <code>sys/generate-root/*</code>	verfügbar	standardmäßig aus seit 2.5.3
<code>raft snapshot inspect</code>	existiert	existiert nicht – nur <code>save</code> und <code>restore</code>
Terraform-Provider	<code>hashicorp/vault</code>	weiterhin <code>hashicorp/vault</code> , auf OpenBao gezeigt

Die drei, die den Aufbau wirklich verändert haben:

Kein APT-Repository. Die Rolle lädt das `.deb` und baut eine eigene Vertrauenskette: gepinnter GPG-Fingerprint → signierte `checksums.txt` → SHA-256 des Pakets. Beide Glieder sind Pflicht; nur die Prüfsumme zu vergleichen hieße, die Datei gegen eine Liste zu prüfen, die ein Angreifer im selben Request hätte tauschen können.

Kein mlock, also kein Swap-Argument. Die Paket-Unit setzt `MemorySwapMax=0`, damit der Prozess auch auf einem Host mit Swap nie ausgelagert wird. Die Rolle prüft vorab, dass der Host `cgroup v2` fährt (sonst wird das Limit stillschweigend ignoriert), und behauptet nach jedem Deployment den *effektiven* Wert – eine künftige Paket-Revision kann es nicht unbemerkt fallen lassen.

Kein snapshot inspect. Das Snapshot-Skript prüft das Archiv selbst: Ein OpenBao-Raft-Snapshot ist ein gzipped tar mit `meta.json`, `state.bin` und einer SHA256SUMS im `coreutils`-Format über die beiden. Das Skript rechnet beide Hashes nach – dieselbe Garantie, die `inspect` gab, ohne Zwischenspeicher.

Teil II – Schicht 1: Ansible

Was die Rolle tut

roles/openbao/	
tasks/preflight.yml	OS, Architektur, cgroup v2, Swap, ausstehender Reboot, NFS-Export
tasks/install.yml	.deb-Download, GPG + SHA-256, dpkg hold
tasks/tls.yml	Key, CSR, selbstsigniertes Zertifikat, SAN-Prüfung
tasks/config.yml	openbao.hcl, Verzeichnisse, logrotate
tasks/service.yml	systemd-Drop-in, MemorySwapMax-Prüfung, Status
tasks/firewall.yml	ufw (SSH zuerst!)
tasks/snapshots.yml	NFS-Mount, Timer, Staleness-Check
tasks/helpers.yml	openbao-unseal, /etc/profile.d

preflight.yml bricht mit Anleitung ab, statt halb zu deployen: fehlender Reboot, falsches OS, cgroup v1, NFS-Export nicht freigegeben – alles wird vor dem ersten apt geprüft.

Die Konfiguration

openbao.hcl ersetzt die Conffile aus dem Paket. Die ausgelieferte nutzt storage "file" – ein Backend ohne Snapshot-Unterstützung. apt läuft mit dem Default force-confold, ein Paket-Upgrade behält also unsere Datei, statt einen interaktiven Prompt zu öffnen, der das Playbook aufhängen würde.

```

ui                = true
log_level         = "info"
log_format        = "json"

# Die Adresse, unter der CLIENTS OpenBao erreichen: der Reverse-Proxy,
# nicht die VM. Sonst zeigen Redirects und die UI ins Leere.
api_addr          = "https://bao.example.internal"
cluster_addr      = "https://192.0.2.40:8201"

# Kein disable_mlock – mit Absicht. OpenBao hat mlock entfernt; der Schutz
# gegen Secrets im Swap ist MemorySwapMax=0 in openbao.service.

storage "raft" {
  path      = "/opt/openbao/data"
  node_id   = "bao-01"
}

listener "tcp" {
  address            = "0.0.0.0:8200"
  cluster_address    = "0.0.0.0:8201"

  tls_cert_file      = "/opt/openbao/tls/openbao.crt"
  tls_key_file        = "/opt/openbao/tls/openbao.key"
  tls_min_version     = "tls12"

  # Seit 2.5.3 Default true: ein unauthentifizierter Aufrufer könnte sonst
  # jede laufende Root-Generierung abbrechen. Break Glass hängt damit am
  # userpass-Admin – siehe Runbook.
  disable_unauthed_generate_root_endpoints = true

  # Ohne dies zeigt JEDER Audit-Eintrag die Proxy-IP statt des Clients.
  # 127.0.0.1 gehört NICHT hinein: x_forwarded_for_reject_not_present ist
  # Default true, und lokale init/unseal/snapshot-Aufrufe senden kein XFF.
  x_forwarded_for_authorized_addrs = "192.0.2.41"
  x_forwarded_for_hop_skips        = "0"
}

# Bewusst NICHT hier: das Audit-Device. Es braucht einen API-Aufruf und damit
# ein Token. Die Rolle bleibt token-frei; Audit kommt aus OpenTofu.

```

Die systemd-Härtung

Ein Drop-in über der Unit aus dem Paket – nur Deltas, die Basis-Unit bleibt unangetastet und überlebt Upgrades:

```

[Service]
# Ersetzt mlock. Die Basis-Unit setzt es auch; hier erneut, damit eine
# künftige Paket-Revision es nicht unbemerkt fallen lässt.
MemorySwapMax=0

# Die Basis-Unit erlaubt CAP_SYSLOG. Wir loggen JSON nach stdout, journald
# übernimmt – keine Capability nötig. Leerer Wert setzt die Liste zurück.
AmbientCapabilities=
CapabilityBoundingSet=

# Ein Core-Dump von OpenBao enthält Klartext-Secrets aus dem Speicher.
LimitCORE=0

# 'strict' deckt zusätzlich /opt und /var ab und braucht die ReadWritePaths.
# Das ERSTE, was man zurücknimmt, wenn OpenBao nach einem Upgrade nicht
# startet: -e openbao_protect_system=full
ProtectSystem=strict
ReadWritePaths=/opt/openbao/data /var/log/openbao

ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictNamespaces=true
RestrictRealtime=true
RestrictSUIDSGID=true
LockPersonality=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX

# Bewusst NICHT gesetzt: MemoryDenyWriteExecute und SystemCallFilter.
# Strenger, aber sie können Plugins brechen (separate Prozesse). Härtung,
# die den Dienst bricht, ist schlechter als etwas weniger Härtung.

```

Deployment

```
ansible-playbook playbooks/deploy-openbao.yml
```

Solange die VM noch nicht in der NFS-Freigabe des NAS steht, bricht snapshots.yml absichtlich mit einer Anleitung ab. Erst ohne NAS deployen, dann nachziehen:

```

# 1) OpenBao in Betrieb nehmen, lokale Snapshots aktiv
ansible-playbook playbooks/deploy-openbao.yml -e openbao_snapshot_nfs_enabled=false

# 2) Nach Freigabe der VM-IP im NFS-Export:
ansible-playbook playbooks/deploy-openbao.yml

```

Der zweite Lauf braucht **keinen Neustart** und damit kein erneutes Unseal – openbao.hcl ändert sich nicht, nur Mount-Unit und Timer kommen dazu.

Danach läuft OpenBao **versiegelt und nicht initialisiert** – der richtige Zustand. bao operator init läuft bewusst nicht durch Ansible, damit Unseal-Keys und Root-Token nie in Task-Ausgaben, -v-Logs oder Terminal-Scrollback landen.

Der NFS-Export

Der Export wird von einem älteren Vault-Knoten **mitbenutzt**. Zwei Systeme, die in ein Verzeichnis schreiben, sind nur sicher, weil die Retention am Dateinamen hängt:

Knoten	Schreibt	Löscht
Vault	vault-<ts>.snap	vault-*.snap
OpenBao	openbao-<ts>.snap	openbao-*.snap

Der Prefix kommt aus openbao_snapshot_prefix und wird an genau zwei Stellen verwendet:

Dateiname und find-Muster in prune(). **Das eine ohne das andere zu ändern heißt, dass dieser Knoten die Backups der anderen Maschine löscht.** Wenn sich diese Kopplung je zu fragil anfühlt, ist die Antwort ein zweiter Export, nicht ein klügeres Muster.

Auf dem NAS: Squash „**root auf admin abbilden**“ für die VM-IP – der Snapshot-Timer läuft als root; ohne passendes Mapping wird root gesquasht und scheitert an der Ordner-ACL. Und: Das Push-Modell bedeutet, wer eine der VMs kompromittiert, kann die Snapshots auf dem NAS löschen – **auch die der anderen**. Schutz dagegen gehört auf die NAS-Seite: Dateisystem-Snapshots oder ein Backup des Exports. Ohne das hilft die zweite Kopie gegen Plattenausfall, nicht gegen Ransomware.

Teil III – Schicht 2: OpenTofu

Warum der Provider vault heißt

Es gibt keinen openbao/openbao-Provider in der Registry. OpenBao hat die Vault-HTTP-API behalten, der HashiCorp-Provider funktioniert unverändert und wird per VAULT_ADDR auf OpenBao gezeigt. Das Risiko: Die Projekte werden divergieren. Wenn ein künftiges Provider-Release Vault-spezifische Versionsstrings oder Enterprise-Endpunkte voraussetzt, die letzte funktionierende Version in `versions.tf` pinnen, statt in den Ressourcen herumzuarbeiten.

Was die Konfiguration anlegt – und warum

Ressource	In OpenBao	Warum
<code>vault_audit.file</code>	Audit-Device nach <code>/var/log/openbao/audit.log</code>	jeder Zugriff wird protokolliert. Ohne das kann man nie beantworten, wer ein Secret gelesen hat
<code>vault_policy.admin</code>	Berechtigungssatz	OpenBao verbietet per Default alles
<code>vault_auth_backend.userpass</code>	Login-Methode	ohne Auth-Methode kommt nur der Root-Token hinein
<code>vault_userpass_auth_backend_user.admin</code>	User admin mit der Policy admin	der persönliche Zugang
<code>vault_policy.snapshot</code>	nur read auf <code>sys/storage/raft/snapshot</code>	der Backup-Timer braucht ein Token, das genau eine Sache darf

Der Zweck in einem Satz: damit der Root-Token widerrufen werden kann. Er gilt unbegrenzt, läuft nie ab und lässt sich nicht einschränken.

Unter OpenBao wiegt dieser Satz schwerer als unter Vault. Die admin-Policy enthält `sys/generate-root-token/*`, weil OpenBao die unauthentifzierten `sys/generate-root/*`-Endpunkte seit 2.5.3 deaktiviert. Drei Unseal-Keys allein erzeugen keinen Root-Token mehr – dieser Login ist der Break-Glass-Weg und muss funktionieren, bevor root verschwindet.

Die Dateien

```
# versions.tf
terraform {
  required_version = ">= 1.11.0"    # write-only attributes ab 1.11

  required_providers {
    vault = { source = "hashicorp/vault", version = "~> 5.0" }
  }

  # State und Plan werden clientseitig verschlüsselt: das Repo liegt in
  # einem Cloud-Sync, und ein unverschlüsselter State mit sensiblen Werten
  # würde versioniert an einen Dritten gehen.
  encryption {
    key_provider "pbkdf2" "main" { passphrase = var.state_passphrase }
    method "aes_gcm" "main"      { keys = key_provider.pbkdf2.main }
    state { method = method.aes_gcm.main }
    plan  { method = method.aes_gcm.main }
  }
}

provider "vault" {
  # Bewusst LEER. Adresse aus VAULT_ADDR, Token aus ~/.vault-token.
  # Ein Token in HCL landet in der Konfiguration UND im State.
}
```

```
# audit.tf – muss ZUERST existieren, damit alles Weitere protokolliert ist.
# Erzwingen nicht durch einen manuellen Schritt, sondern durch depends_on
# an den anderen Ressourcen.
resource "vault_audit" "file" {
  type = "file"
  path = "file"
  options = { file_path = var.audit_log_path }
}
```

```
# auth-userpass.tf
resource "vault_auth_backend" "userpass" {
  type      = "userpass"
  path      = "userpass"
  depends_on = [vault_audit.file]
}

resource "vault_userpass_auth_backend_user" "admin" {
  mount      = vault_auth_backend.userpass.path
  username   = var.admin_username

  # password_wo ist ein WRITE-ONLY-Attribut: der Wert geht an OpenBao, aber
  # NIE in den State oder eine Plan-Datei. Preis: keine Drift-Erkennung.
  # Rotation läuft über das Hochzählen von password_wo_version.
  password_wo      = var.admin_password
  password_wo_version = var.admin_password_version

  token_policies = [vault_policy.admin.name]
  token_ttl      = 3600
  token_max_ttl  = 28800
}
```

```
# variables.tf (Auszug)
variable "admin_password" {
  type          = string
  ephemeral     = true    # während des Laufs benutzt, nirgends persistiert
  # KEIN Default: der Provider verlangt genau eines von password_wo /
  # password_hash_wo – mit null scheitert schon der plan. Die Variable muss
  # also bei JEDEM Lauf gesetzt sein, auch wenn nur eine Policy sich ändert.
}
```

Die admin-Policy im Ganzen – bewusst breit, dieser Login treibt OpenTofu:

```
path "sys/health"           { capabilities = ["read", "sudo"] }
path "sys/capabilities-self" { capabilities = ["update"] }
path "sys/mounts"           { capabilities = ["read", "list"] }
path "sys/mounts/*"         { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
path "sys/auth"             { capabilities = ["read", "list"] }
path "sys/auth/*"           { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "sys/policies/acl"     { capabilities = ["list"] }
path "sys/policies/acl/*"   { capabilities = ["create", "read", "update", "delete", "list"] }
path "sys/audit"            { capabilities = ["read", "list", "sudo"] }
path "sys/audit/*"          { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "sys/leases/*"         { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
path "sys/storage/raft/snapshot" { capabilities = ["read"] }

# Break Glass – OpenBao-spezifisch. Ohne diese Pfade kann dieser Login keinen
# Ersatz-Root-Token erzeugen, und die Unseal-Keys allein auch nicht.
path "sys/generate-root-token/attempt" { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "sys/generate-root-token/update"   { capabilities = ["create", "update", "sudo"] }
path "sys/decode-token"                 { capabilities = ["create", "update"] }

# auth/token/create ist Pflicht: der Provider erzeugt pro Lauf ein
# kurzlebiges Child-Token und braucht update darauf.
path "auth/*"           { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
path "identity/*"       { capabilities = ["create", "read", "update", "delete", "list"] }
path "secret/*"         { capabilities = ["create", "read", "update", "delete", "list"] }
path "kubernetes/*"     { capabilities = ["create", "read", "update", "delete", "list"] }
path "pki/*"            { capabilities = ["create", "read", "update", "delete", "list", "sudo"] }
```

Welcher Wert wohin

Beide Passwörter sind **selbst zu erfinden**. Sie werden nirgends geholt – sie entstehen in dem Moment, in dem man sie zum ersten Mal setzt.

Variable	Was es ist	Was es nicht ist
TF_VAR_state_passphrase	Passphrase, mit der OpenTofu seinen State verschlüsselt . Mindestens 16 Zeichen. Rein lokal, OpenBao sieht sie nie	kein Unseal-Key, kein Root-Token
TF_VAR_admin_password	das persönliche Login-Passwort für OpenBao	nicht das Linux- oder SSH-Passwort
VAULT_ADDR	die Adresse, gelesen vom Provider	–
BAO_ADDR	dieselbe Adresse, gelesen von der bao-CLI	–

Nur eines von beiden zu setzen ist die klassische „geht in tofu, geht nicht in der Shell“-Falle. Ein `env.sh` setzt beide zusammen und lädt die Passwörter aus dem Schlüsselbund des Betriebssystems – ohne sie je anzuzeigen:

```
export VAULT_ADDR="https://bao.example.internal"
export BAO_ADDR="$VAULT_ADDR"
read -rs -p "State passphrase: " TF_VAR_state_passphrase; echo
read -rs -p "OpenBao admin password: " TF_VAR_admin_password; echo
export TF_VAR_state_passphrase TF_VAR_admin_password
```

`read -rs` gibt nichts aus und hinterlässt keine Spur in der History – anders als `export VAR='secret'`, das dauerhaft in `~/.zsh_history` steht.

Warum zufällig statt merkbar: `openssl rand -base64 32` sind 256 Bit Entropie in 44 Zeichen. PBKDF2 ist genau dann verwundbar, wenn die Passphrase selbst schwach ist – Key-Derivation verlangsamt Brute-Force nur um einen Faktor. Bei 256 Bit ist Brute-Force aussichtslos, unabhängig von der Iterationszahl. Man muss sie nie tippen – sie kommt aus dem Schlüsselbund.

Der erste Lauf – mit dem Root-Token

Henne und Ei: Der Zugang, den diese Konfiguration anlegt, existiert noch nicht. Genau **einmal** mit dem Root-Token:

```
cd tofu
source env.sh

# Root-Token ohne Echo lesen statt ihn in die History zu exportieren.
# VAULT_TOKEN, nicht BAO_TOKEN – das liest der Provider.
read -rs -p "Root token: " VAULT_TOKEN; echo; export VAULT_TOKEN

tofu init
tofu plan          # erwartet: 5 to add, 0 to change, 0 to destroy
tofu apply

unset VAULT_TOKEN # ab hier nicht mehr nötig
```

Jeder weitere Lauf

```
source env.sh
bao login -method=userpass username=admin # Token landet in ~/.vault-token
tofu plan
```

VAULT_TOKEN wird bewusst **nicht** gesetzt: Der Provider findet das Token aus `bao login` in `~/.vault-token` selbst. Ein vergessenes VAULT_TOKEN würde es stillschweigend überschreiben.

Der Provider erzeugt **pro Lauf ein kurzlebiges Child-Token** und widerruft es danach. Deshalb erscheinen im Audit-Log bei jedem `tofu plan` zusätzliche Token-Events – erwartetes Verhalten, kein Fehler.

Passwort rotieren

Write-only-Attribute haben keine Drift-Erkennung. Die Änderung muss über den Zähler signalisiert werden:

```
export TF_VAR_admin_password='neues-passwort'
tofu apply -var admin_password_version=2
```

Dann den neuen Wert dauerhaft in `admin_password_version` eintragen.

Teil IV – Das Bootstrap-Runbook

Einmalige Sequenz nach `bao operator init`. **Die Reihenfolge ist nicht beliebig.** Das Audit-Device kommt zuerst, damit jeder folgende Schritt protokolliert ist. Der Root-Token wird zuletzt widerrufen, und nur, nachdem der Ersatzzugang *bewiesen* ist.

Warum Schritt 3 unter OpenBao wichtiger ist als unter Vault. OpenBao deaktiviert die unauthentifzierten `sys/generate-root/*`-Endpunkte, und `bao operator generate-root` nutzt stattdessen die authentifzierten `sys/generate-root-token`-Endpunkte. Alle fünf Unseal-Keys zu halten ist damit allein **kein Weg zurück** – es braucht zusätzlich einen Login mit `sys/generate-root-token/*`, den die `admin`-Policy aus Schritt 2 liefert. Unter Vault war das Überspringen von Schritt 3 verzeihlich. Hier nicht. **Schritt 5 erst, wenn Schritt 3 bestanden ist.**

Voraussetzungen

```
ssh ubuntu@192.0.2.40
bao status          # Initialized true, Sealed false
```

Steht dort `Sealed true`: `sudo openbao-unseal`. Ein `/etc/profile.d`-Snippet setzt `BA0_ADDR` und `BA0_CACERT` für interaktive Shells, sodass `bao` auf der VM ohne Weiteres funktioniert.

1. Mit dem Root-Token anmelden

Genau **einmal**. Der Zugang, der `root` ersetzt, existiert noch nicht.

```
bao login
bao token lookup -format=json | jq -r '.data.policies' # ["root"]
```

2. Konfiguration mit OpenTofu anwenden

Teil III, „Der erste Lauf“. Danach prüfen:

```
tofu output
bao audit list -detailed
bao policy list      # admin, snapshot, default, root
bao auth list        # userpass/ und token/
```

3. Den Ersatzzugang beweisen – nicht überspringen

```
# -token-only ist entscheidend (Kurzform für -field=token -no-store).
# OHNE dieses Flag überschreibt 'bao login' ~/.vault-token und beendet damit
# die laufende Root-Session – BEVOR bewiesen ist, dass der neue Zugang
# funktioniert. So wird aus einer Verifikation ein Lockout.
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"

BAO_TOKEN="$ADMIN_TOKEN" bao token lookup -format=json | jq -r '.data.policies, .data.ttl'
BAO_TOKEN="$ADMIN_TOKEN" bao policy list

# Der Break-Glass-Pfad selbst – das, was Unseal-Keys allein nicht mehr
# können. -init startet eine Root-Generierung, -cancel bricht sie ab: beweist
# die Fähigkeit, ohne einen Root-Token zu erzeugen, den man verwahren müsste.
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel

unset ADMIN_TOKEN
```

Erst wenn admin erscheint, bao policy list funktioniert **und** das generate-root -init/-cancel-Paar durchläuft, ist der Ersatzzugang bewiesen. **Scheitert irgendetwas davon, Schritt 5 unter keinen Umständen ausführen.**

4. Snapshot-Token – der eine Schritt, der manuell bleiben muss

Die Policy kommt aus OpenTofu. Das **Token** nicht: Es ist ein geheimer Wert, der als Datei auf der VM landen muss. vault_token würde ihn in den State schreiben.

```
# -period: ein PERIODISCHES Token. Ein normales läuft ab, und die Snapshots
# hören stillschweigend auf – die häufigste Backup-Fehlerursache überhaupt.
# Jede Nutzung verlängert es, der tägliche Timer hält es also selbst am Leben.
bao token create \
  -policy=snapshot \
  -period=720h \
  -orphan \
  -display-name=openbao-snapshot \
  -field=token \
  | sudo tee /etc/openbao/snapshot.token >/dev/null

sudo chmod 0600 /etc/openbao/snapshot.token
```

-orphan braucht sudo-Capability in OpenBao – geht nur, solange man noch root ist. Prüfen:

```
sudo systemctl start openbao-snapshot.service
sudo journalctl -u openbao-snapshot.service -n 10 --no-pager -o cat
sudo sh -c 'ls -lh /var/backups/openbao/'
```

Erwartet: Snapshot created: /var/backups/openbao/openbao-<ts>.snap. Ohne den NAS-Mount endet der Job mit Fehler – absichtlich: Ein Backup, das die zweite Ablage nie erreicht hat, darf nicht als Erfolg zählen.

5. Root-Token widerrufen

Nur, wenn Schritt 3 bestanden ist – inklusive generate-root-Test.

```
bao token lookup -format=json | jq -r '.data.policies' # ["root"]
bao token revoke -self
bao token lookup # muss jetzt fehlschlagen
```

Ab hier ist userpass der Weg hinein. Break Glass bei verlorenem Zugang:

```
bao login -method=userpass username=admin
bao operator generate-root -init
bao operator generate-root # 3x mit Unseal-Keys
```

Sind Root-Token *und* Admin-Login weg, bleibt als letzter Ausweg, den Legacy-Pfad bewusst und befristet wieder einzuschalten:

```
ansible-playbook playbooks/deploy-openbao.yml \
  -e openbao_disable_unauthed_generate_root_endpoints=false \
  -e openbao_allow_restart=true
ssh ubuntu@192.0.2.40 sudo openbao-unseal
bao operator generate-root -init # geht jetzt ohne Token
```

In derselben Sitzung wieder ausschalten. Es anzulassen ist genau die Exposition, die der Default verhindern soll.

6. Abschlussprüfung

Zuerst: Hat das Snapshot-Token den Root-Widerruf überlebt? Der wichtigste und am leichtesten vergessene Check. Tokens **vererben**: Wer einen Parent widerruft, nimmt die Kinder mit. Genau darum wurde das Snapshot-Token mit `-orphan` erzeugt.

```
sudo systemctl start openbao-snapshot.service # muss mit 0 enden
sudo sh -c 'BAO_TOKEN=$(cat /etc/openbao/snapshot.token) bao token lookup -format=json' \
| jq -r '.data.orphan, .data.policies, .data.period'
```

Erwartet: true, ["default","snapshot"], 2592000. default gehört dazu – das Token braucht sie, um sich selbst zu erneuern. Ist orphan **false**, ist das Token mit root gestorben. Der Weg zurück ist generate-root über den Admin-Login, weil `-orphan` erhöhte Rechte braucht.

Dann:

```
bao login -method=userpass username=admin # hier bewusst MIT Speichern
bao status | grep -E 'Sealed|Initialized' # false / true
bao audit list # file/
bao auth list # userpass/ und token/
bao policy list # admin, snapshot, default, root
systemctl list-timers 'openbao-*' # beide Timer aktiv
sudo sh -c 'ls -lh /var/backups/openbao/' # mindestens ein Snapshot
sudo sh -c 'ls -lh /mnt/snapshots/' # openbao-*.snap neben vault-*.snap
```

Die letzte Zeile aufmerksam lesen: Der Export wird mit dem Vault-Knoten geteilt. Man sollte **beide** Prefixe sehen, und die Anzahl der vault-*.snap darf sich nicht geändert haben.

Teil V – Betrieb

Unseal nach jedem Neustart

```
sudo openbao-unseal
```

Ein kleines Skript, das `bao status` liest, den Fortschritt anzeigt und `bao operator unseal` ohne Argument aufruft – Keys werden interaktiv und ohne Echo gelesen, nie als Argument übergeben (sichtbar über `/proc`, gespeichert in der History). Das ist der bewusst akzeptierte Preis der Shamir-Entscheidung. Solange nichts davon abhängt, ist es nur unbequem. Sobald ein Cluster mit External Secrets daran hängt, wird jeder Stromausfall zum Cluster-Ausfall – dann Auto-Unseal (Transit oder Cloud-KMS) neu bewerten.

Snapshots prüfen

```
systemctl list-timers 'openbao-*'  
sudo sh -c 'ls -lh /var/backups/openbao/ /mnt/snapshots/'  
systemctl status openbao-snapshot-check.service
```

`openbao-snapshot-check.timer` warnt täglich, wenn der letzte **erfolgreiche** Lauf älter als 48 h ist. Es prüft bewusst Erfolg statt Timer – die häufigste Backup-Fehlerursache ist ein abgelaufenes Token, bei dem der Timer weiter feuert und jeder Lauf scheitert.

Einen Snapshot von Hand verifizieren

OpenBao hat kein `bao operator raft snapshot inspect`. Das Archiv ist ein gzipped tar, das geht überall:

```
snap=/mnt/snapshots/openbao-<ts>.snap  
  
tar -tzf "$snap"                                # meta.json, state.bin, SHA256SUMS  
tar -xz0f "$snap" meta.json | jq .              # index, term, version, size  
tar -xz0f "$snap" SHA256SUMS                    # erwartete Hashes  
tar -xz0f "$snap" state.bin | sha256sum         # muss zur Zeile oben passen
```

Das letzte Paar ist genau das, was `openbao-snapshot` nach jedem `save` prüft, bevor die Datei an ihren Platz verschoben wird.

Restore in dieselbe laufende Instanz

Daten versehentlich gelöscht, OpenBao läuft noch – die Unseal-Keys passen, also **ohne** `-force`:

```
bao operator raft snapshot restore /mnt/snapshots/openbao-<ts>.snap  
sudo openbao-unseal                                # danach versiegelt
```

Vorher die Datei mit den tar-Befehlen oben prüfen – es gibt kein inspect, das es für einen tut.

Upgrade

```
ansible-playbook playbooks/deploy-openbao.yml \
  -e openbao_version=2.6.3 -e openbao_allow_restart=true
ssh ubuntu@192.0.2.40 sudo openbao-unseal
```

Das Paket ist per `dpkg hold` fixiert; die Rolle löst den Hold, lädt und verifiziert das neue `.deb`, installiert und setzt den Hold wieder. Die neue Version in **beiden** Stellen eintragen – Inventory und Task-Runner – sonst pinnt das nächste normale deploy den Server zurück.

Teil VI – Disaster Recovery: eine neue VM

Für den Fall, dass die VM nicht mehr existiert: tote Platte, gelöschte VM, korrupte Raft-Daten.

Dieses Verfahren ist unverifiziert, bis man es einmal durchlaufen hat. Der Abschnitt „Restore-Probe“ erklärt, wie das sicher geht. **Jetzt** ist der beste Zeitpunkt: Solange nichts von dieser Instanz abhängt, ist eine Probe risikoarm. Sobald ein Cluster Secrets daraus bezieht, wird sie deutlich heikler.

Was man haben muss

	Wo es liegt	Ohne das ...
Unseal-Keys des ORIGINAL-Clusters (3 von 5)	Passwort-Manager	ist der Snapshot wertlos – er ist damit verschlüsselt
Ein Snapshot openbao-*.snap	NAS	gibt es nichts wiederherzustellen
Das Repository	Git	baut man die VM von Hand
SSH-Key	Arbeitsplatz	kein Zugang zur neuen VM
Das admin-Passwort	Passwort-Manager	siehe unten

Keys und Snapshots dürfen nicht am selben Ort liegen. Liegen die Unseal-Keys auf demselben NAS wie die Snapshots, nimmt ein Verlust beides mit.

Eine OpenBao-spezifische Ergänzung dieser Liste. Unter Vault reichten drei Unseal-Keys, um über die unauthentifizierten Endpunkte einen neuen Root-Token zu erzeugen. OpenBao deaktiviert diese. Nach einem Restore ist der userpass-Admin aus dem Snapshot wieder da – **das Passwort für admin ist damit Teil des Recovery-Materials**, nicht nur ein Komfort. Ist es weg, bleibt nur, die Legacy-Endpunkte bewusst wieder einzuschalten (Runbook, Schritt 5).

Die richtige Datei wählen. vault-*.snap sind die Backups der anderen Maschine und hier nutzlos – sie sind mit einem anderen Key-Satz verschlüsselt. Nur openbao-*.snap gehört zu diesem System.

Das, was alle falsch machen

Ein Restore auf einen **neuen** Cluster braucht **zwei verschiedene Sätze** Unseal-Keys, nacheinander:

- | | |
|---------------------------------|--|
| 1. Neue VM: 'bao operator init' | -> erzeugt NEUE Keys + NEUEN Root-Token |
| 2. Unseal mit den NEUEN Keys | -> OpenBao läuft, aber leer |
| 3. Snapshot einspielen (-force) | -> überschreibt den Keyring aus dem Snapshot |
| 4. Ab jetzt die ORIGINAL-Keys | -> die neuen sind wertlos |

-force ist nötig, weil die Shamir-Keys des frischen Clusters nicht zu den Snapshot-Daten passen, die aus einem anderen Cluster stammen. Nach dem Restore mit den **Original**-Keys entsiegeln, bis der Original-Threshold erreicht ist.

Konsequenz: Keys und Root-Token aus Schritt 1 sind Wegwerf-Material – man braucht sie nur für die Minuten zwischen Schritt 2 und 3. Danach gilt wieder alles aus dem Original: Root-Token, Policies, der User admin mit dem Original-Passwort, Auth-Methoden.

Ablauf

1. Neue VM bereitstellen. Ubuntu 24.04, 1 vCPU / 2 GB / 20 GB. Die IP möglichst wiederverwenden – sonst NFS-Freigabe, Proxy-Upstream und Inventory anpassen. Der Host-Key ändert sich: ssh-keygen -R 192.0.2.40.

2. Ansible-Rolle laufen lassen.

```
ansible-playbook playbooks/deploy-openbao.yml
```

Ergebnis: OpenBao läuft, **nicht initialisiert und versiegelt**. Genau richtig. Der NFS-Mount kommt mit – so holt man den Snapshot im nächsten Schritt. openbao-snapshot.service scheitert vorerst („No snapshot token“) – erwartet, wird in Schritt 7 behoben.

3. Snapshot bereitstellen. Es muss eine **lokale Datei** sein:

```
ssh ubuntu@192.0.2.40
sudo sh -c 'ls -lt /mnt/snapshots/openbao-*.snap | head'
sudo cp /mnt/snapshots/openbao-<ts>.snap /tmp/restore.snap

sudo tar -tzf /tmp/restore.snap
sudo tar -xzf /tmp/restore.snap meta.json | jq .
sudo sh -c 'tar -xzf /tmp/restore.snap SHA256SUMS'
sudo sh -c 'tar -xzf /tmp/restore.snap state.bin | sha256sum'
```

Der Hash aus dem letzten Befehl muss in der SHA256SUMS-Ausgabe neben state.bin stehen. Wenn nicht – oder wenn tar abbricht – die nächstältere Datei nehmen. Das ist der Moment, der entscheidet, ob die 30 Kopien auf dem NAS etwas wert waren.

4. Temporär initialisieren und entsiegeln. Diese Keys sind **Wegwerf-Material**.

```
sudo -i
export BAO_ADDR=https://127.0.0.1:8200

bao operator init -key-shares=1 -key-threshold=1    # 1 reicht, sie sind temporär
bao operator unseal <temp-key>
export BAO_TOKEN=<temp-root-token>
bao status          # Sealed false
```

5. Snapshot einspielen.

```
bao operator raft snapshot restore -force /tmp/restore.snap
```

-force ist **Pflicht**: Die Shamir-Keys dieses Clusters passen nicht zu den Daten im Snapshot, und ohne das Flag verweigert OpenBao genau deswegen. Danach versiegelt es sich selbst, weil der Keyring ersetzt wurde.

6. Mit den ORIGINAL-Keys entsiegeln.

```
bao operator unseal      # 3x, mit den Keys des ORIGINAL-Clusters
bao status               # Sealed false, Total Shares 5, Threshold 3
```

Ab hier ist alles zurück: Policies, Audit-Konfiguration, userpass, der User admin mit dem Original-Passwort, der Original-Root-Token.

7. Nacharbeiten. Ein neues Snapshot-Token erzeugen – das alte existiert wieder in den Daten, aber sein Wert lebte nur in der Datei auf der alten VM:

```
bao login -method=userpass username=admin
bao token create -policy=snapshot -period=720h -orphan \
  -display-name=openbao-snapshot -field=token \
  | sudo tee /etc/openbao/snapshot.token >/dev/null
sudo chmod 0600 /etc/openbao/snapshot.token
sudo systemctl start openbao-snapshot.service
sudo rm -f /tmp/restore.snap
```

OpenTofu abgleichen – der State beschreibt die Lage vor dem Ausfall:

```
cd tofu && source env.sh
bao login -method=userpass username=admin
tofu plan      # erwartet: "No changes"
```

Meldet plan Änderungen, ist der Snapshot älter als die letzte Konfigurationsänderung. Dann tofu apply – genau dafür ist die Konfiguration als Code gepflegt.

8. Prüfen.

```
bao status          # Initialized true, Sealed false, Version 2.6.2
bao audit list      # file/
bao auth list       # userpass/
bao policy list     # admin, snapshot, default, root
bao login -method=userpass username=admin # das Original-Passwort muss gehen
systemctl list-timers 'openbao-*'
sudo sh -c 'ls -lh /var/backups/openbao/ /mnt/snapshots/'
curl -s -o /dev/null -w '%{http_code}\n' https://bao.example.internal/v1/sys/health # 200
```

Auch die NAS-Liste auf Kollateralschäden prüfen: Die Anzahl der vault-*.snap muss unverändert sein. Ein wiederhergestellter Knoten mit falschem openbao_snapshot_prefix würde beim ersten Timer-Lauf die Backups der anderen Maschine löschen.

Restore-Probe

Nicht auf der Produktions-VM üben. Eine zweite VM aufsetzen und das Verfahren durchlaufen – mit einer zusätzlichen Regel:

Die Probe-VM muss netzwerk-isoliert sein. Eine wiederhergestellte Instanz hält jede Lease des Originals. Läuft sie parallel und ist erreichbar, kann sie Drittsystem-Credentials widerrufen und damit Leases der echten Instanz ungültig machen.

Praktisch: die Probe-VM in einem isolierten Netz oder ohne NIC booten, Zugang nur über die Konsole; den Snapshot als Datei übertragen; Proxy und Firewall nicht anfassen; mit `-e openbao_snapshot_nfs_enabled=false` deployen – eine Probe-VM darf den geteilten NFS-Export **nie** mounten, ihr Snapshot-Timer würde dort `openbao-*.snap` gegen *ihre* Retention löschen.

Kann ich wirklich wiederherstellen? – Checkliste

Beantwortbar ohne Ausfall:

- ☐ Liegen die 5 Unseal-Keys im Passwort-Manager, **nicht** auf dem NAS?
- ☐ Liegt das Passwort für admin im Passwort-Manager? (Unter OpenBao ist es Recovery-Material.)
- ☐ Ist der neueste `openbao-*.snap` auf dem NAS jünger als 24 h?
- ☐ Ist er lesbar? `tar -tzf` plus der SHA256SUMS-Check aus Schritt 3
- ☐ Überlebt das NAS den Verlust der VM? (ja – separates Gerät)
- ☐ Überlebt es einen Angreifer, der die VM verschlüsselt? **Derzeit nein** – die VM darf Dateien auf dem NAS löschen. Dateisystem-Snapshots auf dem NAS würden die Lücke schließen.
- ☐ Habe ich das Verfahren **mindestens einmal** durchlaufen?

Teil VII – Bewusste Entscheidungen und bekannte Risiken

Entscheidungen

Kein `disable_mlock` – OpenBao hat mlock entfernt, die Option ist obsolet. Secrets bleiben per `MemorySwapMax=0` aus dem Swap; Preflight prüft cgroup v2, `service.yml` behauptet den effektiven Wert nach jedem Lauf.

.deb mit eigener Vertrauenskette – kein APT-Repo, also keine Repo-Signatur, die apt prüfen könnte. Gepinnter Fingerprint → signierte `checksums.txt` → SHA-256. Download nur, wenn sich die Version wirklich ändert.

`disable_unauthed_generate_root_endpoints` bleibt `true` – der sichere Default. Der Preis: Break Glass rückt in die Bootstrap-Reihenfolge. Erst den userpass-Admin *beweisen*, dann root widerrufen.

Kein `notify: restart openbao` – ein Neustart **versiegelt**. Ein naiver Handler würde OpenBao beim zweiten Playbook-Lauf vom Netz nehmen und versiegelt zurücklassen. Bewusst freischalten: `-e openbao_allow_restart=true`.

`x_forwarded_for_authorized_addrs` ohne `127.0.0.1` – `x_forwarded_for_reject_not_present` ist Default `true`. Mit localhost in der Liste würden lokale `init/unseal`-Aufrufe (ohne XFF-Header) abgewiesen.

SANs mit localhost und `127.0.0.1` – sonst braucht jeder lokale bao-Aufruf `-tls-skip-verify`. Die falsche Gewohnheit beim Entsiegeln eines Secrets-Stores.

systemd-Timer statt eingebauter Snapshot-Automatik – `sys/storage/raft/snapshot-auto` ist ein Vault-Enterprise-Feature, das OpenBao ebenfalls nicht hat.

MemoryDenyWriteExecute und SystemCallFilter nicht gesetzt – strenger, aber sie können Plugins brechen. Härtung, die den Dienst bricht, ist schlechter als etwas weniger Härtung.

Risiken

Risiko	Status
Versiegelt nach jedem Neustart, bis 3 Keys eingegeben sind	bewusst akzeptiert
Unseal-Keys allein sind kein Weg zurück – Break Glass braucht einen funktionierenden Login	durch Runbook-Reihenfolge + sys/generate-root-token/* in der Admin-Policy entschärft
Geteilter NFS-Export: ein falsches Prune-Muster löscht die Backups des Vault-Knotens	Prefix-verankertes find; an drei Stellen dokumentiert
Push-Backup: wer eine VM kompromittiert, kann die NAS-Snapshots beider Systeme löschen	Schutz gehört auf die NAS-Seite
Write-Back-Cache ohne gesunde Controller-Batterie ⇒ Raft/BoltDB-Korruption bei Stromausfall	Controller prüfen, USV klären
Ein Knoten, ein Host	Schutz durch Restore, nicht durch Quorum
Der hashicorp/vault-Provider zielt auf Vault, nicht auf OpenBao	funktioniert heute; letzte gute Version pinnen, wenn ein Release Vault-Spezifika voraussetzt
Restore nie getestet	siehe Teil VI

Troubleshooting

Symptom	Ursache	Abhilfe
Proxy antwortet 400 , Body Client sent an HTTP request to an HTTPS server.	Upstream steht auf http, der Listener spricht TLS	Upstream auf https://192.0.2.40:8200
Proxy antwortet 502	OpenBao läuft nicht oder ist unerreichbar	systemctl status openbao, ufw-Regel für 192.0.2.41 prüfen
400 mit leerem Body von OpenBao	XFF-Header fehlt, x_forwarded_for_reject_not_present ist true	Proxy muss X-Forwarded-For setzen
501	erreichbar, aber nicht initialisiert	erwartet vor bao operator init
Error loading CA File: permission denied	BA0_CACERT zeigt nach /opt/openbao/tls (0750 root:openbao)	die weltlesbare Kopie unter /usr/local/share/ca-certificates/ nutzen
Snapshot-Service failed , Copy to /mnt/snapshots failed	NAS-Verzeichnis zeigt Modus 000, root wird gesquasht	NFS-Freigabe: Squash „root auf admin“ für die VM-IP
Snapshot-Service failed, lokaler Snapshot existiert	Beabsichtigt. Ein Backup, das die zweite Ablage nicht erreicht hat, zählt nicht	NAS-Zugriff reparieren
Snapshot failed its integrity check	Archiv nicht entpackbar oder Hashes stimmen nicht	Datei wurde absichtlich verworfen; Platte und Journal prüfen, Service erneut starten
ls: cannot access '/var/backups/openbao/*.snap' trotz sudo	der Glob wird vor sudo expandiert, als normaler User ohne Leserecht	sudo sh -c 'ls /var/backups/openbao/'
Service startet nach Upgrade nicht	ProtectSystem=strict blockiert einen Pfad, den die neue Version schreibt	-e openbao_protect_system=full, dann den Pfad finden
tofu: Failed to request input for var.state_passphrase	TF_VAR_state_passphrase nicht gesetzt; der encryption-Block wird statisch ausgewertet, ein Prompt ist unmöglich	source env.sh
tofu apply: 403 permission denied mitten im Lauf	Token abgelaufen (TTL 1 h) – oder ~/.vault-token gehört zur anderen Instanz	bao token lookup, dann bao token renew oder neu anmelden

Anhang A – Alle Kommandos

```
# — Schicht 1: Ansible —————
ansible-playbook playbooks/deploy-openbao.yml -e openbao_snapshot_nfs_enabled=false
ansible-playbook playbooks/deploy-openbao.yml # nach NFS-Freigabe

# — init / unseal (auf der VM, normales Terminal) —————
export BAO_ADDR=https://127.0.0.1:8200
bao operator init -key-shares=5 -key-threshold=3 # Ausgabe -> Passwort-Manager
sudo openbao-unseal

# — Schicht 2: OpenTofu, erster Lauf —————
bao login # Root-Token, einmalig
cd tofu && source env.sh
read -rs -p "Root token: " VAULT_TOKEN; echo; export VAULT_TOKEN
tofu init && tofu plan && tofu apply
unset VAULT_TOKEN

# — Ersatzzugang beweisen —————
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"
BAO_TOKEN="$ADMIN_TOKEN" bao policy list
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel
unset ADMIN_TOKEN

# — Snapshot-Token —————
bao token create -policy=snapshot -period=720h -orphan \
  -display-name=openbao-snapshot -field=token | sudo tee /etc/openbao/snapshot.token >/dev/
null
sudo chmod 0600 /etc/openbao/snapshot.token
sudo systemctl start openbao-snapshot.service

# — Root widerrufen (erst nach bestandenem Test) —————
bao token revoke -self

# — Betrieb —————
sudo openbao-unseal
systemctl list-timers 'openbao-*'
tar -xzOf /mnt/snapshots/openbao-<ts>.snap state.bin | sha256sum
bao operator raft snapshot restore /mnt/snapshots/openbao-<ts>.snap # gleiche Instanz
bao operator raft snapshot restore -force /tmp/restore.snap # neue VM
```

Anhang B – Glossar

Shamir 5/3 – Der Master-Key ist in fünf Anteile zerlegt, drei beliebige rekonstruieren ihn. Nach jedem Neustart müssen drei eingegeben werden.

Raft – Integriertes Storage-Backend. Auch mit einem Knoten sinnvoll, weil nur Raft snapshot save/restore kann.

Root-Token – Aus `init`. Unbegrenzt gültig, nicht einschränkbar. Nach dem Bootstrap widerrufen.

Break Glass – Der Weg zu einem neuen Root-Token. Unter OpenBao: Unseal-Keys **und** ein Login mit `sys/generate-root-token/*`.

Periodisches Token – Ein Token mit `-period`, das sich bei jeder Nutzung um diese Spanne verlängert. Für Timer-Jobs, die nie ablaufen dürfen.

Orphan-Token – Ein Token ohne Parent. Überlebt den Widerruf des Tokens, das es erzeugt hat.

Write-only-Attribut (`password_wo`) – Ein Terraform/OpenTofu-Attribut, das an den Provider geht, aber nie in State oder Plan geschrieben wird.

Conffile – Eine Konfigurationsdatei, die ein Debian-Paket mitbringt und bei Upgrades besonders behandelt. `force-confold` behält die lokale Version.

MemorySwapMax=0 – cgroup-v2-Limit, das einem Prozess Swap verbietet. Ersetzt unter OpenBao das entfernte `mlock`.

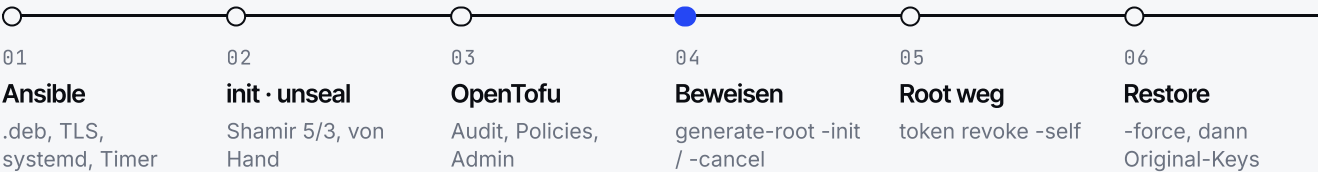
XFF – X-Forwarded-For, der Header, mit dem ein Reverse-Proxy die Client-IP weitergibt. Ohne Konfiguration zeigt das Audit-Log nur den Proxy.

Ein Secrets-Store muss zwei Dinge gleichzeitig: sicher sein und wiederkommen.

Dieser Leitfaden baut OpenBao als einzelnen Knoten auf einer VM – Raft, Shamir, TLS, systemd-Härtung, tägliche Snapshots auf ein NAS. Ansible installiert, OpenTofu konfiguriert, und die wenigen Schritte, die manuell bleiben müssen, stehen als Runbook da.

Mit dem, was OpenBao anders macht als Vault: kein APT-Repo, kein mlock, kein snapshot inspect, keine unauthentifzierten generate-root-Endpunkte. Und einem Disaster Recovery auf eine leere VM, das erklärt, warum dabei zwei Sätze Unseal-Keys nacheinander gebraucht werden.

```
-- Die Reihenfolge, die einen Lockout ausschließt
tofu apply                                # Audit, Policies, Admin
bao operator generate-root -init          # Break Glass beweisen
bao operator generate-root -cancel
bao token revoke -self                   # erst jetzt
```



ubuntu	24.04 LTS · 1 vCPU · 2 GB
openbao	2.6.2 · .deb · raft
ansible	2.17 · Schicht 1
opentofu	≥ 1.11 · Schicht 2 · verschl. State
backup	systemd-Timer · NFS

33 Seiten · 7 Teile · 2 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.
CC BY 4.0 · Weitergabe erlaubt · ohne Gewähr